SCHOOL MANAGEMENT SYSTEM — DATABASE MANAGEMENT

🔹 DATABASE CREATION (DDL)

Command Type: DDL — Data Definition Language

```
CREATE DATABASE school_mgmt;
USE school_mgmt;
```

🔹 CORE TABLE DESIGN (DDL)

Command Type: DDL

STUDENT TABLE

```
CREATE TABLE student (
student_id INT PRIMARY KEY,
name VARCHAR(50) NOT NULL,
class_id INT,
phone BIGINT CHECK (phone LIKE '9%' OR phone LIKE '8%' OR phone LIKE
'7%')
);
```

DDL Used: CREATE, PRIMARY KEY, CHECK

FACULTY TABLE

```
CREATE TABLE faculty (
faculty_id INT PRIMARY KEY,
faculty_name VARCHAR(50),
department VARCHAR(50)
);
```

📚 SUBJECT TABLE

```
CREATE TABLE subject (
subject_id INT PRIMARY KEY,
subject_name VARCHAR(50),
faculty_id INT,
FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
);
```

DDL Used: FOREIGN KEY

🏫 CLASSROOM TABLE

```
CREATE TABLE classroom (
class_id INT PRIMARY KEY,
class_name VARCHAR(20),
capacity INT
);
```

📝 MARKS TABLE

```
CREATE TABLE marks (
student_id INT,
subject_id INT,
marks INT CHECK (marks BETWEEN 0 AND 100),
PRIMARY KEY (student_id, subject_id),
FOREIGN KEY (student_id) REFERENCES student(student_id),
FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
);
```

📅 ATTENDANCE TABLE

```
CREATE TABLE attendance (
student_id INT PRIMARY KEY,
total_days INT,
present_days INT,
FOREIGN KEY (student_id) REFERENCES student(student_id)
);
```

```
mysql> USE school_mgmt;
Database changed
mysql> CREATE TABLE student (
    ->     student_id INT PRIMARY KEY,
    ->     name VARCHAR(50) NOT NULL,
    ->     class_id INT,
    ->     phone BIGINT CHECK (phone LIKE '9%' OR phone LIKE '8%' OR phone LIKE '7%')
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE faculty (
    ->     faculty_id INT PRIMARY KEY,
    ->     faculty_name VARCHAR(50),
    ->     department VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> CREATE TABLE subject (
    ->     subject_id INT PRIMARY KEY,
    ->     subject_name VARCHAR(50),
    ->     faculty_id INT,
    ->     FOREIGN KEY (faculty_id) REFERENCES faculty(faculty_id)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE classroom (
    ->     class_id INT PRIMARY KEY,
    ->     class_name VARCHAR(20),
    ->     capacity INT
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE marks (
    ->     student_id INT,
    ->     subject_id INT,
    ->     marks INT CHECK (marks BETWEEN 0 AND 100),
    ->     PRIMARY KEY (student_id, subject_id),
    ->     FOREIGN KEY (student_id) REFERENCES student(student_id),
    ->     FOREIGN KEY (subject_id) REFERENCES subject(subject_id)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE TABLE attendance (
    ->     student_id INT,
    ->     total_days INT,
    ->     present_days INT,
    ->     PRIMARY KEY (student_id),
    ->     FOREIGN KEY (student_id) REFERENCES student(student_id)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO faculty VALUES
    -> (1,'Dr. Kumar','Maths'),
    -> (2,'Ms. Priya','Physics'),
```

🔹 INSERTING MASTER DATA (DML)

Command Type: DML — Data Manipulation Language

Faculty

```
INSERT INTO faculty VALUES
(1,'Dr. Kumar','Maths'),
(2,'Ms. Priya','Physics'),
(3,'Mr. Rahul','Chemistry'),
(4,'Mrs. Anita','Computer Science');
```

Subjects

```
INSERT INTO subject VALUES
(101,'Mathematics',1),
(102,'Physics',2),
(103,'Chemistry',3),
(104,'Computer Science',4);
```

Classroom

```
INSERT INTO classroom VALUES (1,'10-A',40);
```

Students

```
INSERT INTO student VALUES
(1,'Arjun',1,9876543210),
(2,'Sneha',1,9876543211),
(3,'Rahul',1,9876543212),
(4,'Priya',1,9876543213),
(5,'Amit',1,9876543214),
(6,'Neha',1,9876543215),
(7,'Karan',1,9876543216),
(8,'Meena',1,9876543217),
(9,'Suresh',1,9876543218),
(10,'Anita',1,9876543219);
```

```
mysql> CREATE TABLE attendance (
    ->     student_id INT,
    ->     total_days INT,
    ->     present_days INT,
    ->     PRIMARY KEY (student_id),
    ->     FOREIGN KEY (student_id) REFERENCES student(student_id)
    -> );
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO faculty VALUES
    -> (1,'Dr. Kumar','Maths'),
    -> (2,'Ms. Priya','Physics'),
    -> (3,'Mr. Rahul','Chemistry'),
    -> (4,'Mrs. Anita','Computer Science');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO subject VALUES
    -> (101,'Mathematics',1),
    -> (102,'Physics',2),
    -> (103,'Chemistry',3),
    -> (104,'Computer Science',4);
Query OK, 4 rows affected (0.00 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO classroom VALUES (1,'10-A',40);
Query OK, 1 row affected (0.00 sec)

mysql> INSERT INTO student VALUES
    -> (1,'Arjun',1,9876543210),
    -> (2,'Sneha',1,9876543211),
    -> (3,'Rahul',1,9876543212),
    -> (4,'Priya',1,9876543213),
    -> (5,'Amit',1,9876543214),
    -> (6,'Neha',1,9876543215),
    -> (7,'Karan',1,9876543216),
    -> (8,'Meena',1,9876543217),
    -> (9,'Suresh',1,9876543218),
    -> (10,'Anita',1,9876543219);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO marks VALUES
    -> -- Topper
    -> (1,101,95),(1,102,92),(1,103,94),(1,104,96),
    ->
    -> -- Average (60–85)
    -> (2,101,75),(2,102,68),(2,103,72),(2,104,80),
    -> (3,101,65),(3,102,70),(3,103,60),(3,104,78),
    ->
    -> -- Fail in one subject
    -> (4,101,82),(4,102,30),(4,103,75),(4,104,70),
    ->
    -> -- Others (mixed)
    -> (5,101,55),(5,102,62),(5,103,58),(5,104,60),
```

Marks (Realistic Distribution)

```
INSERT INTO marks VALUES
(1,101,95),(1,102,92),(1,103,94),(1,104,96),
(2,101,75),(2,102,68),(2,103,72),(2,104,80),
(3,101,65),(3,102,70),(3,103,60),(3,104,78),
(4,101,82),(4,102,30),(4,103,75),(4,104,70),
(5,101,55),(5,102,62),(5,103,58),(5,104,60),
(6,101,88),(6,102,79),(6,103,81),(6,104,85),
(7,101,40),(7,102,45),(7,103,38),(7,104,42),
(8,101,72),(8,102,75),(8,103,78),(8,104,80),
(9,101,33),(9,102,67),(9,103,70),(9,104,60),
(10,101,90),(10,102,88),(10,103,85),(10,104,89);
```

Attendance

```
INSERT INTO attendance VALUES
(1,200,190),(2,200,170),(3,200,160),(4,200,150),
(5,200,145),(6,200,180),(7,200,130),(8,200,165),
(9,200,120),(10,200,185);
```

```
    -> (10,'Anita',1,9876543219);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> INSERT INTO marks VALUES
    -> -- Topper
    -> (1,101,95),(1,102,92),(1,103,94),(1,104,96),
    ->
    -> -- Average (60–85)
    -> (2,101,75),(2,102,68),(2,103,72),(2,104,80),
    -> (3,101,65),(3,102,70),(3,103,60),(3,104,78),
    ->
    -> -- Fail in one subject
    -> (4,101,82),(4,102,30),(4,103,75),(4,104,70),
    ->
    -> -- Others (mixed)
    -> (5,101,55),(5,102,62),(5,103,58),(5,104,60),
    -> (6,101,88),(6,102,79),(6,103,81),(6,104,85),
    -> (7,101,40),(7,102,45),(7,103,38),(7,104,42),
    -> (8,101,72),(8,102,75),(8,103,78),(8,104,80),
    -> (9,101,33),(9,102,67),(9,103,70),(9,104,60),
    -> (10,101,90),(10,102,88),(10,103,85),(10,104,89);
Query OK, 40 rows affected (0.01 sec)
Records: 40  Duplicates: 0  Warnings: 0

mysql> INSERT INTO attendance VALUES
    -> (1,200,190),(2,200,170),(3,200,160),(4,200,150),
    -> (5,200,145),(6,200,180),(7,200,130),(8,200,165),
    -> (9,200,120),(10,200,185);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0

mysql> SELECT student_id, AVG(marks) AS average_marks
    -> FROM marks
    -> GROUP BY student_id;
+------------+---------------+
| student_id | average_marks |
+------------+---------------+
|          1 |       94.2500 |
|          2 |       73.7500 |
|          3 |       68.2500 |
|          4 |       64.2500 |
|          5 |       58.7500 |
|          6 |       83.2500 |
|          7 |       41.2500 |
|          8 |       76.2500 |
|          9 |       57.5000 |
|         10 |       88.0000 |
+------------+---------------+
10 rows in set (0.00 sec)

mysql> SELECT student_id
    -> FROM marks
    -> GROUP BY student_id
    -> HAVING MIN(marks) >= 90;
+------------+
```

🔹 INFORMATIVE QUERIES (DQL)

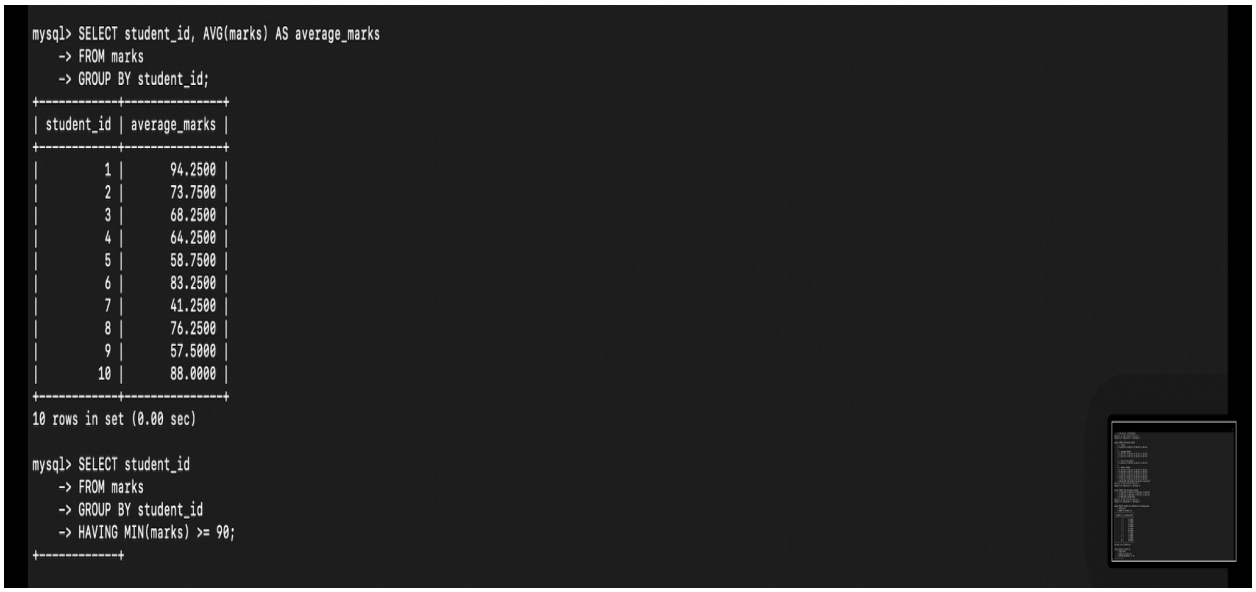
Command Type: DQL — Data Query Language

Average Marks per Student

```
SELECT student_id, AVG(marks) AS average_marks
FROM marks
GROUP BY student_id;
```

Toppers (All Subjects ≥ 90)

```
SELECT student_id
FROM marks
GROUP BY student_id
HAVING MIN(marks) >= 90;
```



```
mysql> SELECT student_id, AVG(marks) AS average_marks
    -> FROM marks
    -> GROUP BY student_id;
+------------+--------------+
| student_id | average_marks |
+------------+--------------+
|          1 |      94.2500 |
|          2 |      73.7500 |
|          3 |      68.2500 |
|          4 |      64.2500 |
|          5 |      58.7500 |
|          6 |      83.2500 |
|          7 |      41.2500 |
|          8 |      76.2500 |
|          9 |      57.5000 |
|         10 |      88.0000 |
+------------+--------------+
10 rows in set (0.00 sec)

mysql> SELECT student_id
    -> FROM marks
    -> GROUP BY student_id
    -> HAVING MIN(marks) >= 90;
+------------+
```

Failed Students

```
SELECT DISTINCT student_id
FROM marks
WHERE marks < 35;
```

```
mysql> SELECT student_id
    -> FROM marks
    -> GROUP BY student_id
    -> HAVING MIN(marks) >= 90;
+------------+
| student_id |
+------------+
|          1 |
+------------+
1 row in set (0.00 sec)

mysql> SELECT DISTINCT student_id
    -> FROM marks
    -> WHERE marks < 35;
+------------+
| student_id |
+------------+
|          4 |
|          9 |
+------------+
2 rows in set (0.00 sec)
```

Faculty–Subject–Student Mapping

```
SELECT s.name, sub.subject_name, f.faculty_name, m.marks
FROM student s
JOIN marks m ON s.student_id = m.student_id
JOIN subject sub ON m.subject_id = sub.subject_id
JOIN faculty f ON sub.faculty_id = f.faculty_id;
```

```
mysql> CREATE VIEW Student_Full_Report AS
    -> SELECT s.name, sub.subject_name, f.faculty_name, m.marks
    -> FROM student s
    -> JOIN marks m ON s.student_id = m.student_id
    -> JOIN subject sub ON m.subject_id = sub.subject_id
    -> JOIN faculty f ON sub.faculty_id = f.faculty_id;
Query OK, 0 rows affected (0.01 sec)
```

Attendance Shortage (<75%)

```
SELECT student_id
FROM attendance
WHERE (present_days / total_days) * 100 < 75;
```

```
mysql> USE school_mgmt;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT student_id
    -> FROM attendance
    -> WHERE (present_days / total_days) * 100 < 75;
+------------+
| student_id |
+------------+
|          5 |
|          7 |
|          9 |
+------------+
3 rows in set (0.00 sec)
```

PART 2: ADVANCED OPERATIONS & AUTOMATION

🔹 TASK 4: ADVANCED DATA MANIPULATION (Session 4)

These commands fulfill the requirement for complex updates and conditional data handling.

1. Update Phone Numbers with Country Code

In a professional system, we often need to prepend country codes.

```
SQL
UPDATE student SET phone = CONCAT('+91-', phone);
-- Verification
SELECT name, phone FROM student LIMIT 3;
```

```
mysql> SELECT name, phone FROM student LIMIT 3;
+-------+------------+
| name  | phone      |
+-------+------------+
| Arjun | 9876543210 |
| Sneha | 9876543211 |
| Rahul | 9876543212 |
+-------+------------+
3 rows in set (0.00 sec)

mysql>
```

2. Conditional Grade Update (Using CASE)

This shows how to manipulate data based on conditions.

SQL

```
SELECT student_id, marks,
CASE
    WHEN marks >= 90 THEN 'A+'
    WHEN marks >= 75 THEN 'A'
    WHEN marks >= 50 THEN 'B'
    ELSE 'C'
END AS Grade
FROM marks;
```

```
Database changed
mysql> SELECT student_id, marks,
    -> CASE
    ->     WHEN marks >= 90 THEN 'A+'
    ->     WHEN marks >= 75 THEN 'A'
    ->     WHEN marks >= 50 THEN 'B'
    ->     ELSE 'C'
    -> END AS Grade
    -> FROM marks;
+------------+-------+-------+
| student_id | marks | Grade |
+------------+-------+-------+
|          1 |    95 | A+    |
|          1 |    92 | A+    |
|          1 |    94 | A+    |
|          1 |    96 | A+    |
|          2 |    75 | A     |
|          2 |    68 | B     |
|          2 |    72 | B     |
|          2 |    80 | A     |
|          3 |    65 | B     |
|          3 |    70 | B     |
|          3 |    60 | B     |
|          3 |    78 | A     |
|          4 |    82 | A     |
|          4 |    30 | C     |
|          4 |    75 | A     |
|          4 |    70 | B     |
|          5 |    55 | B     |
|          5 |    62 | B     |
|          5 |    58 | B     |
|          5 |    60 | B     |
|          6 |    88 | A     |
|          6 |    79 | A     |
|          6 |    81 | A     |
|          6 |    85 | A     |
|          7 |    40 | C     |
|          7 |    45 | C     |
|          7 |    38 | C     |
|          7 |    42 | C     |
|          8 |    72 | B     |
|          8 |    75 | A     |
|          8 |    78 | A     |
|          8 |    80 | A     |
|          9 |    33 | C     |
|          9 |    67 | B     |
|          9 |    70 | B     |
|          9 |    60 | B     |
|         10 |    90 | A+    |
|         10 |    88 | A     |
|         10 |    85 | A     |
|         10 |    89 | A     |
+------------+-------+-------+
40 rows in set (0.00 sec)
```

🔹 TASK 5: PROGRAMMABILITY & INTEGRITY (Session 5)

This section covers User-Defined Functions (UDFs) and Database Views.

1. Creating a Database View (Security)

Views allow you to hide complex joins from the end user.

SQL
:
```
SELECT * FROM Student_Full_Report WHERE marks < 40;
```

```
mysql> SELECT * FROM Student_Full_Report WHERE marks < 40;
+--------+--------------+--------------+-------+
| name   | subject_name | faculty_name | marks |
+--------+--------------+--------------+-------+
| Suresh | Mathematics  | Dr. Kumar    |    33 |
| Priya  | Physics      | Ms. Priya    |    30 |
| Karan  | Chemistry    | Mr. Rahul    |    38 |
+--------+--------------+--------------+-------+
3 rows in set (0.00 sec)

mysql>
```

2. User Defined Function (UDF)

Create a function that calculates the "Attendance Percentage."

```
SQL
DELIMITER //
CREATE FUNCTION GetAttendancePercentage(present INT, total INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC
BEGIN
    RETURN (present / total) * 100;
END //
DELIMITER ;

-- Use it in a query:
SELECT student_id, GetAttendancePercentage(present_days, total_days)
AS Attendance_Percentage
FROM attendance;
```

```
mysql> SELECT student_id, GetAttendancePercentage(present_days, total_days) AS Attendance_Percentage
    -> FROM attendance
    -> ;
+------------+----------------------+
| student_id | Attendance_Percentage |
+------------+----------------------+
|          1 |                95.00 |
|          2 |                85.00 |
|          3 |                80.00 |
|          4 |                75.00 |
|          5 |                72.50 |
|          6 |                90.00 |
|          7 |                65.00 |
|          8 |                82.50 |
|          9 |                60.00 |
|         10 |                92.50 |
+------------+----------------------+
10 rows in set (0.03 sec)

mysql>
```